
Algorithm Complexity, Recursion

- ❑ Algorithm Complexity
- ❑ Recursion

Algorithm Complexity

Introduction

- ▶ Algorithms need to be analyzed for their efficiency, in terms of
 - ▶ Running time → computational complexity
 - ▶ Size of used memory → memory complexity
- ▶ An algorithm may run faster/slower and use more/less on certain data sets than on others
 - many indicators for assessing the efficiency:
 - ▶ Average case
 - ▶ Best case (lower bound)
 - ▶ Worst case (upper bound)
 - ▶ Most common case
- ▶ How to measure complexity?
 - ▶ Experimental studies
 - ▶ Theoretical analysis with pseudo-code, flowcharts

Big-O notation

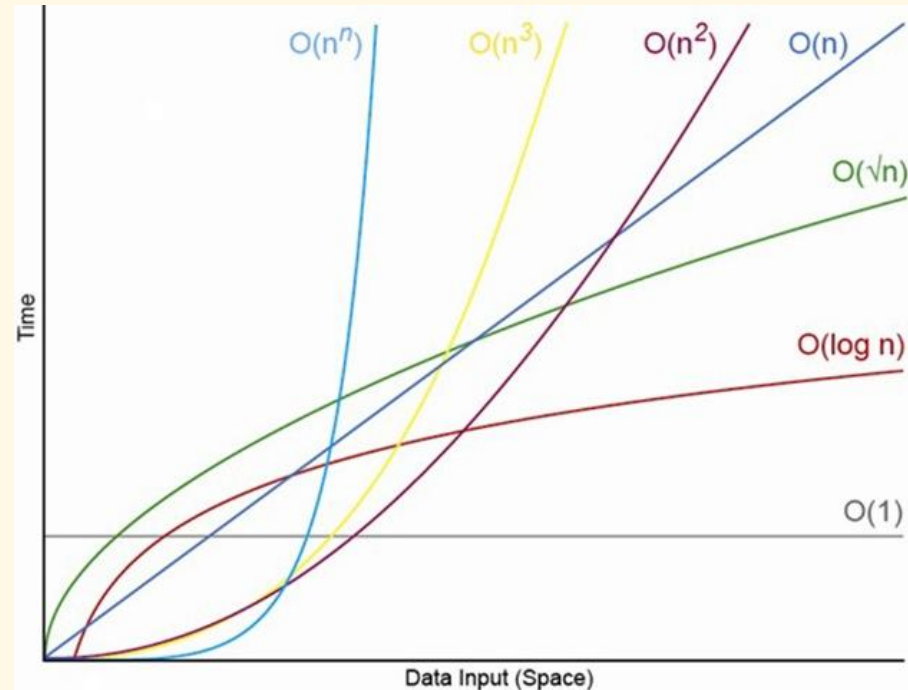
- ▶ Definition: Suppose $f(n)$ and $g(n)$ are non-negative functions of n . Then we say that $f(n)$ is $O(g(n))$ if there exists constants $C > 0$ and $N > 0$ such that for all $n > N$, $f(n) \leq Cg(n)$.
- ▶ This says that function $f(n)$ grows at a rate no faster than $g(n)$, thus $g(n)$ is an upper bound on $f(n)$
- ▶ Big-O expresses an upper bound on the growth rate of a function, for sufficiently large values of n
 - ▶ It represents the computational/memory complexity of algorithms

Examples

- ▶ For $f(n) = 3n + 5$ and $g(n) = n$ there are positive constants C and N such that $f(n) \leq Cg(n)$ for $n > N$
→ $3n + 5$ is $O(n)$
- ▶ $3n^2 + 5n + 4$ is $O(n^2)$
- ▶ $n + \sqrt{n}$ is $O(n)$
- ▶ $2^n + n^2$ is $O(2^n)$

Common Classes

- ▶ Constant: $O(1)$
- ▶ Linear: $O(n)$
- ▶ Quadratic: $O(n^2)$
- ▶ Polynomial: $O(n^k)$, $k \geq 1$
- ▶ Exponential: $O(a^n)$, $n > 1$
- ▶ Logarithmic: $O(\log n)$
- ▶ Factorial: $O(n!)$



- ▶ Efficiency comparison of classes
 - ▶ $O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(3^n) < O(n!) < O(n^n)$

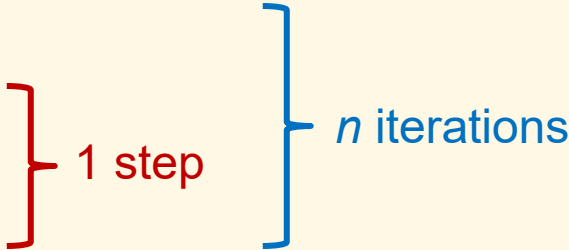
Asymptotic Notation

- ▶ It is correct to say $3n + 2$ is $O(n^3)$, $O(2^n)$
 - ➔ One should make approximation as tight as possible
 - ➔ Asymptotic notation
- ▶ Simple rule: Drop lower order terms and constant factors
 - ▶ $3n + 2$ is $O(n)$
 - ▶ $8n^2 \log n + 5n^2 + n$ is $O(n^2 \log n)$

Computational Complexity Analysis

- ▶ Break down into number of primitive operations
- ▶ Example 1: Find the maximum element of an array.
 - ▶ Algorithm `ArrayMax(A, n)`:
Input: Array A storing n integers.
Output: The maximum element in A .

```
currentMax  $\leftarrow$   $A[0]$ 
for  $i \leftarrow 1$  to  $n-1$  do
    if currentMax  $<$   $A[i]$  then
        currentMax  $\leftarrow$   $A[i]$ 
return currentMax
```



Computational Complexity Analysis (*cont'd*)

► Example 2: Compute prefix averages.

► Algorithm PrefixAverages(X, n):

Input: Array X of n numbers.

Output: Array A of n numbers such that $A[i]$ is the average of $X[0..i]$.

Initialize A as array of n numbers.

for $i \leftarrow 0$ to $n-1$ do

$a \leftarrow 0$

 for $j \leftarrow 0$ to i do

$a \leftarrow a + X[j]$

$A[i] \leftarrow a/(i+1)$

} 1 step

} i iterations
($i = 0..n-1$)

} n iterations

return A

Computational Complexity Analysis (*cont'd*)

- ▶ Example 3: Compute prefix averages.
 - ▶ Algorithm BetterPrefixAverages(X, n):
Input: Array X of n numbers.
Output: Array A of n numbers such that $A[i]$ is the average of $X[0..i]$.

Initialize A as array of n numbers.

$a \leftarrow 0$

for $i \leftarrow 0$ to $n-1$ do

$a \leftarrow a + X[i]$

$A[i] \leftarrow a/(i+1)$

} 1 step } n iterations

return A

Memory Complexity Analysis

- ▶ Revisit above examples and determine the memory complexity

Recursion

Problems Defined Recursively

- ▶ There are many problems whose solution can be defined recursively
- ▶ Example 1: factorial function

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ 1 \times 2 \times 3 \times \dots \times (n-1) \times n & \text{if } n > 0 \end{cases} \quad (\text{closed-form solution})$$

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ (n-1)! \times n & \text{if } n > 0 \end{cases} \quad (\text{recursive solution})$$

Recursive Function

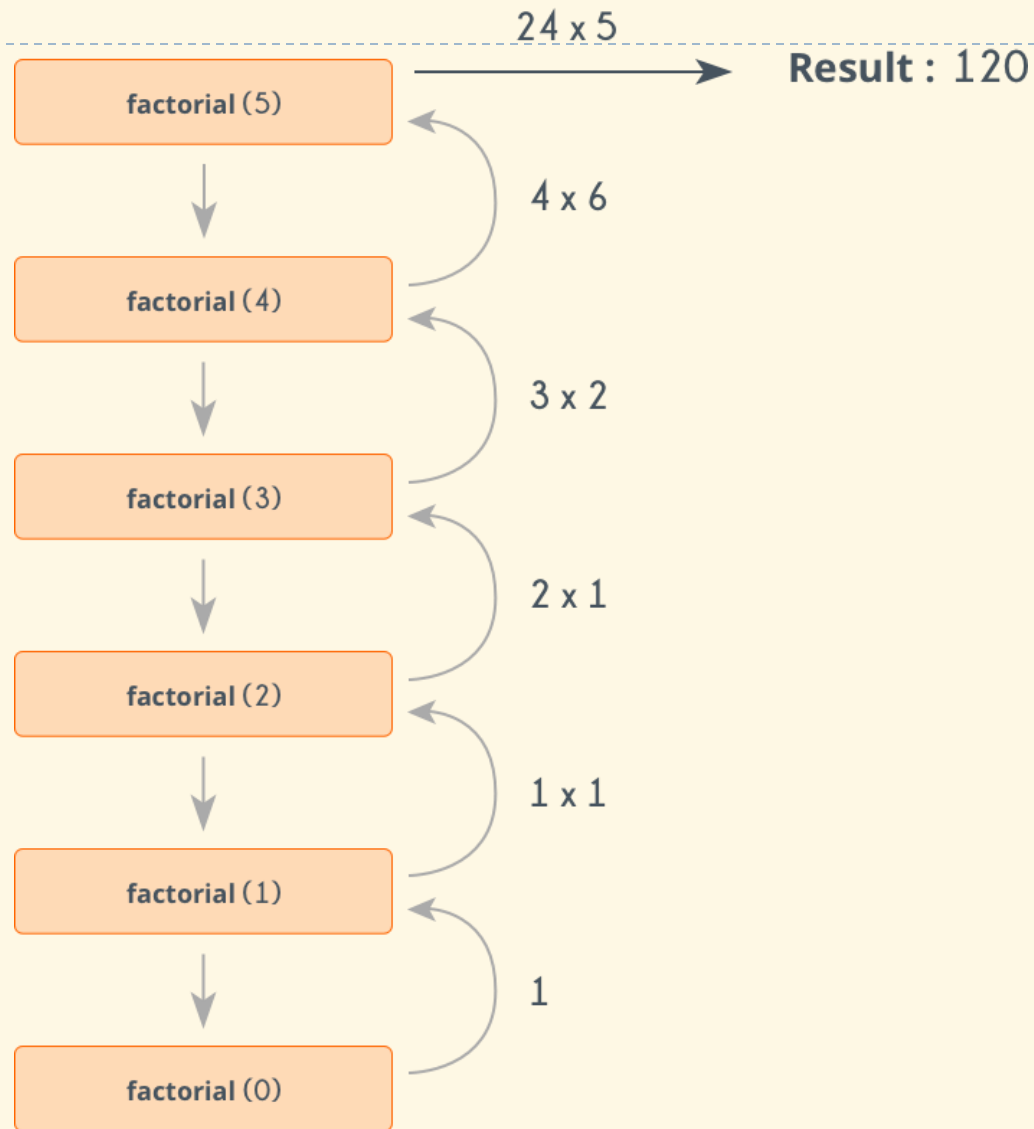
- ▶ A function that calls itself, either in direct or indirect manner
- ▶ Implementation

```
double factorialIterative(int n) {  
    double f = 1;  
    for (int i = 1; i <= n; i++)  
        f *= i;  
    return f;  
}
```

```
double factorialRecursive(int n) {  
    return n == 0 ? 1. : n * factorialRecursive(n-1);  
}
```

```
printf("5! = %lf\n", factorialIterative(5));  
printf("5! = %lf\n", factorialRecursive(5));
```

Explanation



Example 2

- ▶ Calculate x^n , $n \in \mathbb{N}$
- ▶ Implementation

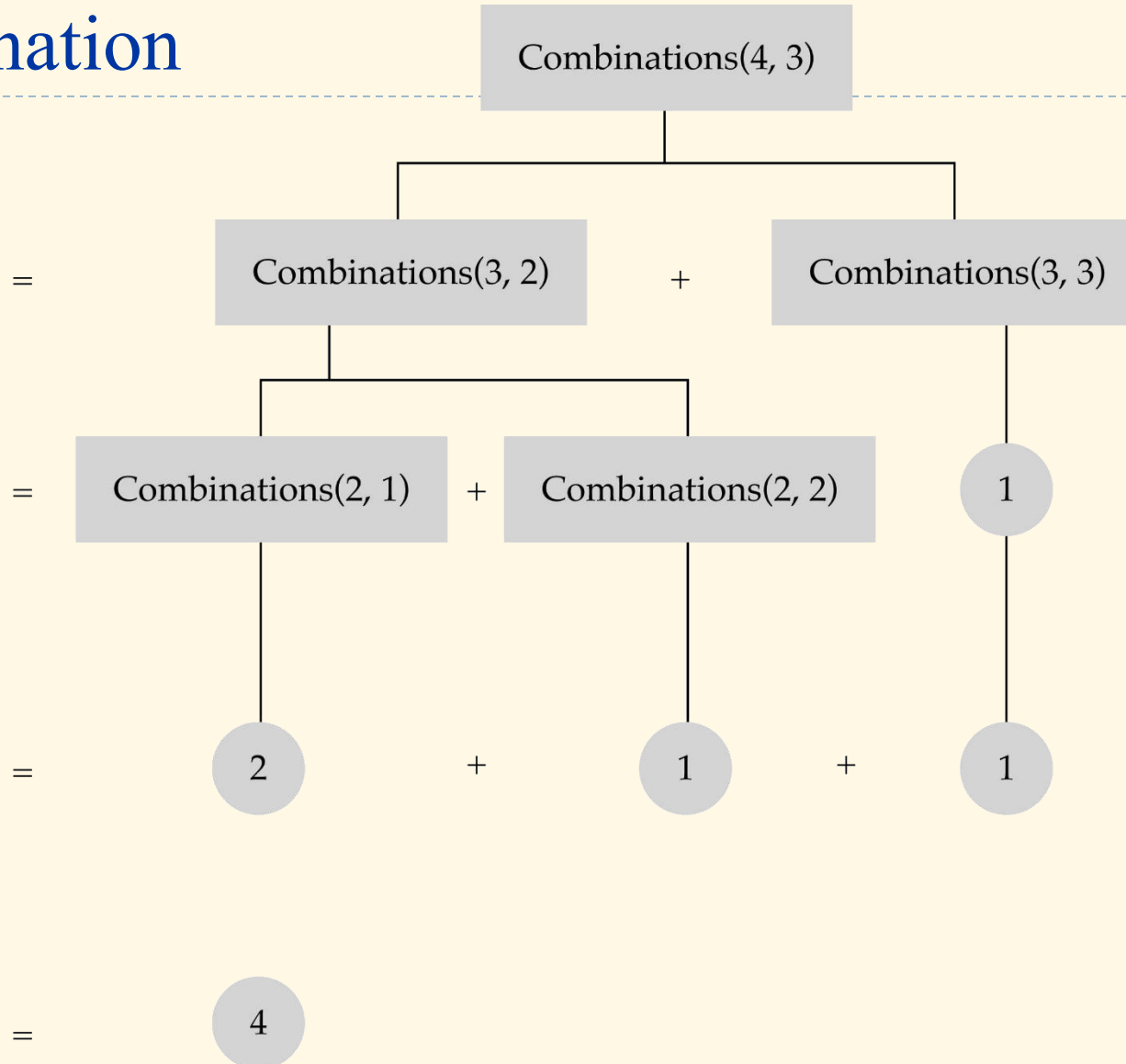
```
double powerIterative(double x, int n) {  
    double y = 1;  
    for (int i = 1; i <= n; i++)  
        y *= x;  
    return y;  
}
```

```
double powerRecursive(double x, int n) {  
    if (n == 0) return 1.;  
    double y = powerRecursive(x, n/2);  
    return (n % 2 == 0) ? y*y : y*y*x;  
}
```

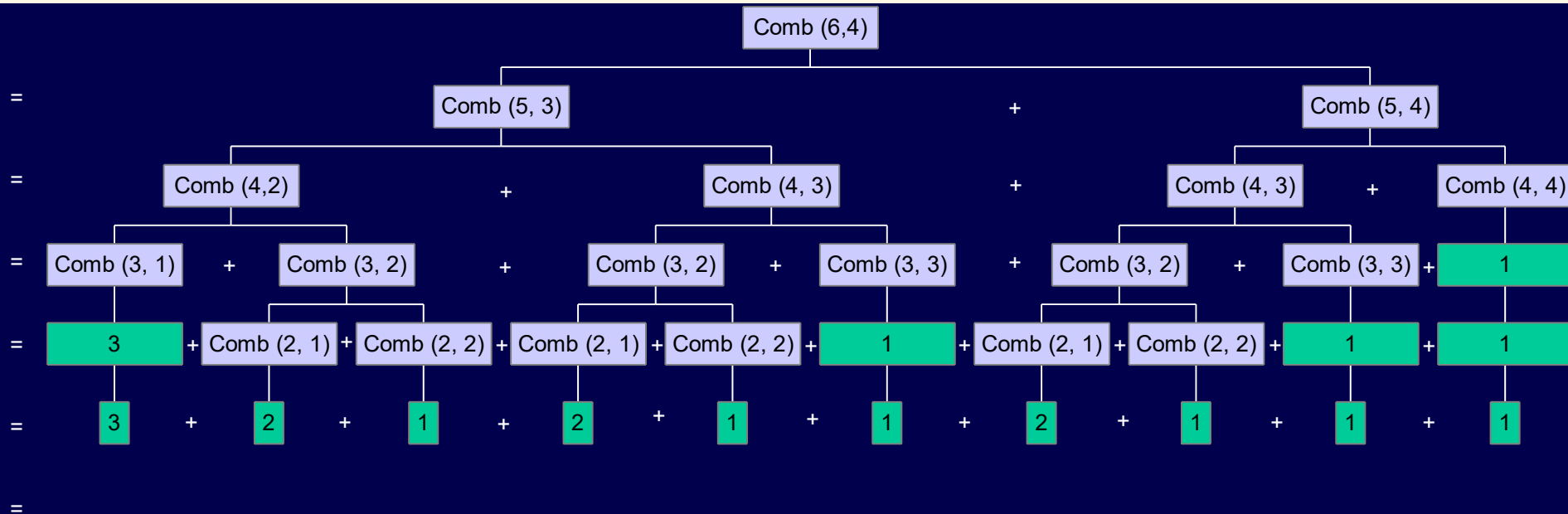

Example 3

- ▶ Calculate number of combinations of k elements from n .
- ▶ Closed-form solution:
 - ▶ $C_n^k = \frac{n!}{k!(n-k)!}$
- ▶ Recursive solution using Pascal's formula:
 - ▶ $C_n^k = \frac{n-k+1}{k} C_n^{k-1}$ with base case $C_n^1 = n$
- ▶ Binary recursive solution:
 - ▶ $C_n^k = C_{n-1}^{k-1} + C_{n-1}^k$ with base cases $C_n^1 = n$ and $C_n^n = 1$

Explanation



Recursion Can be Very Inefficient in Some Cases



Recursion vs. Iteration

- ▶ Iteration can be used in place of recursion
 - ▶ An iterative algorithm uses a looping construct
 - ▶ A recursive algorithm uses a branching structure
- ▶ Recursive solutions are often less efficient, in terms of both time and space, than iterative solutions
- ▶ Recursion can simplify the solution of a problem, often resulting in shorter, more easily understood source code

How to Write a Recursive Function?

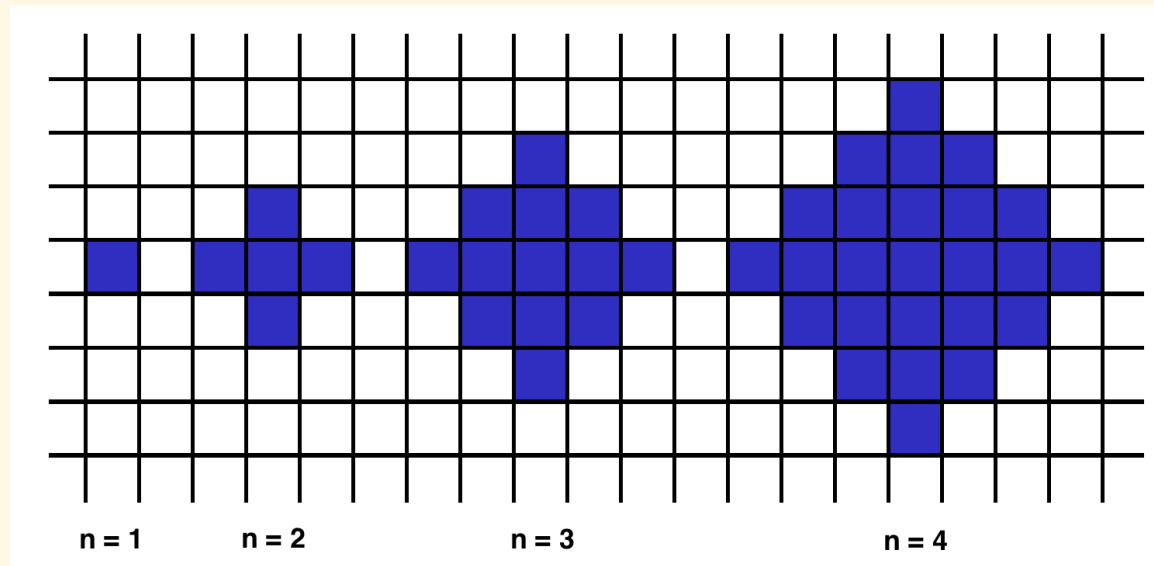
- ▶ Determine the size factor
- ▶ Determine the base case(s) (the one for which you know the answer)
- ▶ Determine the general case(s) (the one where the problem is expressed as a smaller version of itself)
- ▶ Verify the algorithm

Exercises

1. Write the pseudo-code to express the algorithm to look-up a word from a dictionary.
 - ▶ Function `WordLookup( $D$ ,  $W$ )`
 - Input: Dictionary D as an array of N words already sorted in alphabetical order.
 - Word W to be looked-up.
 - Output: Index of W found in D , or -1 if not found.
 - ... (to do)
2. Analyze the complexity of above examples.

Exercises

3. The so-called n -interesting polygon is defined as in the picture. Write a recursive function to find the area of a polygon for a given n .



Exercises

4. The Tower of Hanoi is a puzzle consisting of 3 rods and n disks. The task is to move all disks to another rod, but: only the uppermost disk can be moved, and larger disks cannot be placed on top of smaller disks. Write a recursive function to solve the problem given n .

